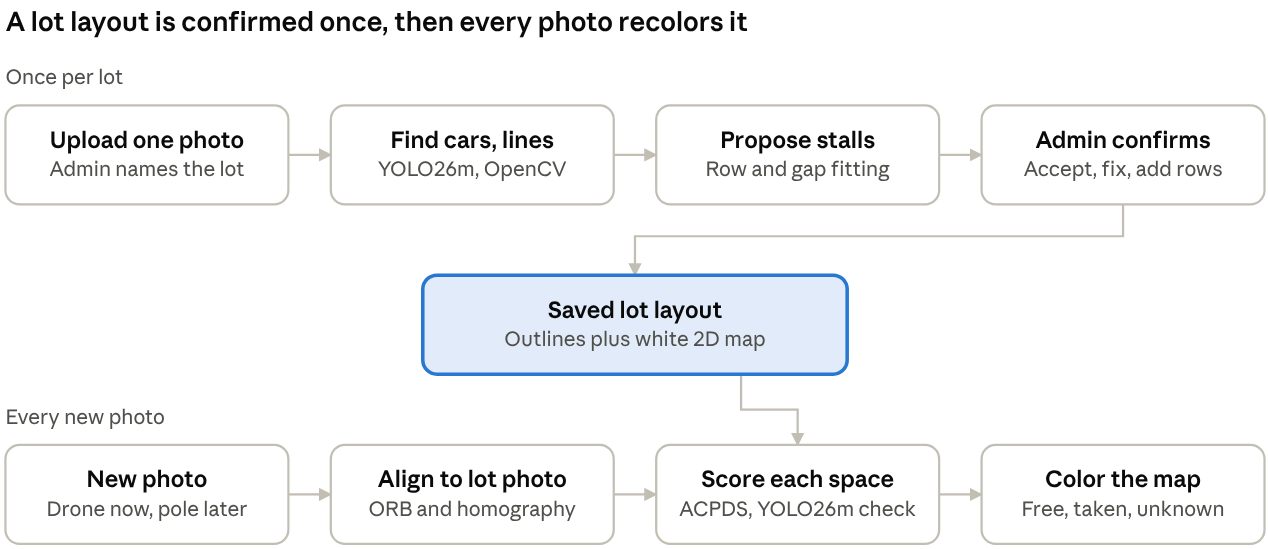
Parking detection and lot onboarding plan

Oct 15, 2026 · @Nathan

The app scores every mapped space with ACPDS on CPU, uses YOLO26m as a safety net for missed cars, and onboards a new lot from one photo that an admin confirms. Both models run unchanged on a laptop and in the AWS worker Lambda. A labeled pilot settles the thresholds by Oct 23, 2026.

At a glance



lot onboarding and photo scoring · 9 steps

Onboarding runs once per lot and ends with a layout an admin has confirmed. Every later photo only aligns, scores and recolors that saved map.

Model stack

ACPDS makes the open or taken call for every space. Everything else either feeds it outlines or double-checks it.

Tool	Job	Score it gives	License	Developer and data origin	Runs on
ACPDS	Decides open or taken for every mapped space on every photo	Occupied score from 0 to 1 per space	MIT	ParkDots and PosAm, Slovakia. 293 GoPro photos from about 10 m up a pole	Laptop and Lambda, CPU
YOLO26m	Finds parked cars at onboarding, then acts as a safety net for cars ACPDS misses	Confidence per car box	AGPL-3.0, fine for a public course repo	Ultralytics, US. COCO photos	Laptop and Lambda, CPU, about 220 ms per 640 px pass
OpenCV ORB plus homography	Lines up each new photo with the lot's reference photo	Match quality, used to reject bad photos	Apache-2.0	OpenCV project	Laptop and Lambda
OpenCV line detector	Finds painted stall lines at onboarding	Line strength	MIT, in OpenCV 4.5.4 or later	OpenCV project	Laptop and Lambda
Row and gap fitting	Proposes stall outlines between and beside parked cars	Evidence score per proposed stall	Team code	Team	Laptop and Lambda
OpenAI Decisions API	Optional cloud alternative to ACPDS, tested in the pilot only	Probability from 0 to 1 per space	Public beta, $0.10 per 1M input tokens	OpenAI, US	AWS only, needs an OpenAI key

Five ACPDS details fail without an error if missed. Build the model with `RCNN(roi_res=128, pooling_type='square')`, pass corners as fractions of the image (x divided by width minus 1), convert OpenCV's BGR to RGB, read output index 1 as occupied, and mirror the weights file to the team's S3 bucket.

How a space gets its state

A space is FREE only when both models agree it is empty. Either model alone can mark it OCCUPIED.

State	When	Map color
OCCUPIED	ACPDS score above the high cutoff, or a confident YOLO26m car box covers 30% or more of the space	Red with a car icon
FREE	ACPDS score below the low cutoff, no YOLO26m car box touches the space beyond a small margin (about 5%), and the whole space is in the photo	Green outline
UNKNOWN	Everything else, including in-between scores, a car over the line (US-11 #2), trees or glare (US-12 #3) and spaces cut off by the frame	Gray with a question mark
CLOSED	An active admin closure, applied by the API when it builds the status	Barrier icon

The pilot sets both cutoffs. The ACPDS score is stored as the space's confidence for the snapshot detail (US-06 #6). A photo that fails alignment writes nothing, and every space keeps its previous state and time (US-12 #1).

Adding a new lot

The system proposes the whole layout from one photo, and an admin confirms it before it is saved. No ready-made model finds empty stalls reliably, so the proposal leans on parked cars and painted lines.

1. The admin selects Add a lot, enters the name and allowed data age, and uploads one photo from the lot's camera position. A photo taken when the lot is busy works best, since every parked car anchors a stall.
2. YOLO26m finds the parked cars, and the OpenCV line detector finds painted stall lines.
3. Row and gap fitting groups the cars into rows, measures stall width and angle, and proposes an outline for every stall, including empty ones in gaps and at row ends. Each proposal carries an evidence score.
4. The admin reviews the proposals on the mapping screen. They accept all, delete extras, drag a corner, or add a missing row with the row tool (four clicks around the row plus a stall count).

5. The system flattens the confirmed outlines to a top-down view, labels rows A, B, C and stalls A-01 onward, and saves the white 2D map as layout version 1.
6. ACPDS scores the onboarding photo, so the new lot shows real colors immediately. Any stall it calls occupied where YOLO26m saw no car is flagged for a second look.

Two things stay manual. A fully empty row with faded paint gives the system nothing to find, so the admin adds it with the row tool. The confirm step also stays, because a wrong outline is wrong on every photo after it. Later, accumulated photos let the system suggest stalls where cars keep appearing but no outline exists.

Drivers pick a lot from the lot list (US-15) and see its white map with each space colored by state.

Camera setup

The drone acts as a pole camera until installed cameras replace it. That view matches what ACPDS learned from and what a real camera will see.

- Hover about 8 to 12 m up with the camera angled down, from a marked takeoff point. Record altitude, heading and gimbal angle so every flight repeats.
- Pick positions where a real pole or mount could go, and use the fewest positions that see every space.
- Skip straight-down shots from high up, since no installed camera will see the lot that way.
- Swapping in a fixed camera needs no code change. It uploads to the same API with lot, viewpoint and capture time.
- Expect more UNKNOWN in far rows where cars hide each other.
- FAA and university drone rules still apply (HLD section 10).

Ruled out

Twelve options were dropped for fit, license or origin. PKLot stays as early test data, since it comes from fixed pole cameras, is CC BY 4.0 and labels every space.

Option	Why it was dropped
RANGER (YOLO11n, car and free classes)	Hong Kong publisher, undisclosed training data, non-commercial license. Technically a good fit, so it can rejoin the pilot if the origin rule is relaxed
NVIDIA LocateAnything-3B	NVIDIA GPU only, no per-box confidence, non-commercial, built on Alibaba's Qwen2.5 and Moonshot AI's MoonViT
UniqueData parking dataset	30 sample images, CC BY-NC-ND 4.0, full set is paid
Ultralytics DOTA rotated-box and VisDrone models	Overhead training data from Chinese universities (Wuhan and Tianjin)
TypeSafe Jev	No image input yet, early access waitlist
Amazon Rekognition	Needs AWS credentials, so it can't run on a laptop (US-14). Comparison only
Esri Parking Spot Detection	Needs ArcGIS, returns boxes only, built for 7 to 10 cm per pixel imagery
AccessParkCV	Finds only disability and aisle-adjacent spaces
Car-camera slot detectors (DMPR-PS, GCN-parking-slot)	Trained on China-origin ps2.0 data, output entrance points rather than outlines
Open-vocabulary detectors (OWLv2, Grounding DINO, YOLO-World)	Weak on aerial imagery, best F1 was 0.276
SAM 3	Prompts for a parking spot return the whole lot or an aisle, not single stalls
Gemini polygon output	Too imprecise to accept without review. Optional experiment only

Pilot by Oct 23

No public test covers any of these models on drone parking photos, so the team's own labeled photos make the final call. Nothing is trained. Only cutoffs, image size and matching rules get tuned.

Setups to run on the same photos

- ACPDS alone
- ACPDS with YOLO26m required to agree before FREE
- YOLO26m alone, as a baseline
- OpenAI Decisions API per space, only if the team allows sending crops to OpenAI

Metric	How it is computed	Pass mark
False-free rate (NFR-3)	Truly occupied spaces reported FREE, divided by all truly occupied spaces	5% or less
Per-space accuracy (NFR-4)	Correct FREE or OCCUPIED calls divided by spaces a human can judge, with UNKNOWN counted as wrong	90% or more
UNKNOWN rate	UNKNOWN calls on spaces a human can judge	10% or less (suggested cap)
Car over the line and hard-to-see spaces	Neighbors of straddling cars and spaces flagged as unclear	Never FREE
Worker time	Alignment through status write, in Docker capped at 2 CPUs and 4 GB	p95 of 30 s or less
Peak memory	Same container	Under 3 GB

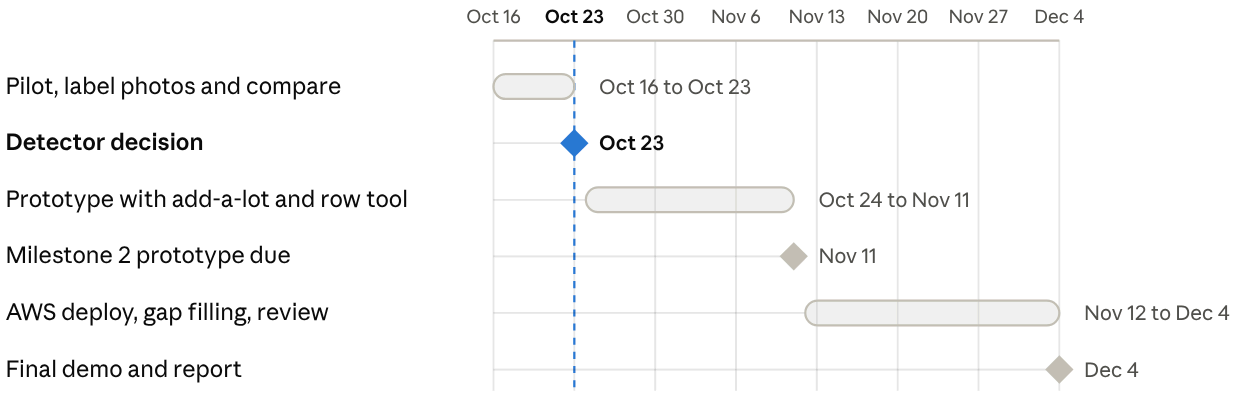
Use 20 to 30 photos covering every camera position and three times of day, with at least 300 occupied spaces labeled. That keeps a measured 5% false-free rate within about 2.5 points. Tune on one capture session and score on another. Run PKLot first to test the code end to end before drone photos exist.

Also run the outline test. Shift the hand-drawn outlines by 5, 10, 20 and 30% of a stall's width and see when ACPDS's answers start changing. That sets how precise auto-proposed outlines need to be.

Schedule

The detector is chosen Oct 23, 19 days before the prototype is due

CS 498 parking app plan by week, fall 2026



semester plan · 3 phases, 3 deadlines

The pilot gets one week, so labeling should start as soon as the first flights are done. The prototype runs locally, and AWS deployment follows in the last phase.

Doc updates

High-Level Design v0.2 and the Milestone 1 stories need nine changes to match these decisions.

- [ ] HLD section 4.4, make ACPDS the primary model, YOLO26m the safety net and the Decisions API optional, replacing LocateAnything-3B
- [ ] HLD section 4.1 step 4, a failed alignment writes nothing and keeps prior states, matching US-12 #1 and US-06 #2
- [ ] HLD section 1.2, move multiple lots into scope to match US-15 and US-16
- [ ] HLD section 1.3, define a viewpoint as a virtual pole camera position
- [ ] HLD section 10, note that space crops go to OpenAI if the Decisions API is used
- [ ] US-06 #6, show the ACPDS score as confidence and make overlap optional
- [ ] US-07 and US-08, add the row tool and the review step for proposed outlines
- [ ] New US-17 Onboard a New Lot From One Photo, with Given-When-Then criteria
- [ ] NFR-4, state that UNKNOWN counts as wrong

Open questions

- Can the AWS version send space crops to OpenAI's Decisions API, or does everything stay self-hosted?
- Does RANGER rejoin the pilot if the origin rule covers developers only, not training data?
- Is a clean top-down map the right driver view? It is the current default.
- Is a 10% UNKNOWN cap acceptable to the team and the instructor?

Sources

The full research behind this plan is in the project docs Drop in parking detector options and Automatic parking space mapping.

- ACPDS repo and paper
- Ultralytics YOLO26 docs and license
- OpenAI Decisions API guide
- TypeSafe Jev announcement
- NVIDIA LocateAnything-3B model card
- RANGER parking model
- PKLot on Hugging Face
- UniqueData parking dataset
- SAM 3 parking spot example
- Seo and Urmson, parking lot structure from aerial images (CMU)
- Grbić and Koch, parking slot detection from vehicle history
- Open-vocabulary detectors on aerial imagery (NTUA 2026)